

Ejercicio SGA Web con JPA



SGA Web con JPA y Jakarta EE

Implementación de EJBs con JPA y Jakarta EE en sga-jee-web

En esta guía, configuraremos el proyecto **sga-jee-web** para trabajar con **EJBs** y **JPA** en **Glassfish** con **Jakarta EE**.

1 Detener el Servidor GlassFish

Antes de realizar cualquier cambio en el código, asegurémonos de que GlassFish no está en ejecución:

1. **Abrir NetBeans.**
2. **Ir a** Services -> Servers.
3. **Click derecho** sobre GlassFish Server.
4. **Seleccionar** Stop si está en ejecución.

2 Agregar Configuración de Persistencia

Necesitamos mover la configuración de persistencia desde el proyecto **holamundo-jpa** a **sga-jee-web**.

1. **Copiar el archivo** `persistence.xml`

desde `holamundo-jpa/src/main/resources/META-INF`

hacia `sga-jee-web/src/main/resources/META-INF`

3 Modificar la Clase de Entidad `Persona.java`

Reemplazaremos la clase **Persona.java** del proyecto `holamundo-jpa` en `sga-jee-web`.

✂ Código actualizado y final:

```
package sga.domain;

import java.io.Serializable;
import jakarta.persistence.*;

@Entity
@NamedQueries({
    @NamedQuery(name="Persona.findAll",
        query="SELECT p FROM Persona p ORDER BY p.idPersona")
})
@Table(name="persona")
public class Persona implements Serializable{
    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    @Column(name="id_persona")
    private int idPersona;

    private String nombre;

    private String apellido;

    private String email;

    private String telefono;

    public Persona() {
    }

    public Persona(String nombre, String apellido, String email, String telefono) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.email = email;
        this.telefono = telefono;
    }

    public int getIdPersona() {
```

```
        return idPersona;
    }

    public void setIdPersona(int idPersona) {
        this.idPersona = idPersona;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getTelefono() {
        return telefono;
    }

    public void setTelefono(String telefono) {
        this.telefono = telefono;
    }

    @Override
    public String toString() {
        return "Persona{" + "idPersona=" + idPersona + ", nombre=" + nombre + ", " +
        apellido=" + apellido + ", email=" + email + ", telefono=" + telefono + '}';
    }
}
```

✓ Cambios:

- Se agregaron `@NamedQueries` para consultas predefinidas en JPA.

4 Modificar `PersonaServiceImpl.java`

Temporalmente, modificamos el método `listarPersonas()` para retornar `null` y evitar errores.

✦ Código actualizado:

```
@Override
public List<Persona> listarPersonas() {
    return null; // Temporalmente, hasta que implementemos la capa de datos
}
```

5 Agregar Dependencias de `holamundo-jpa` a `sga-jee-web`

Ahora, agregamos las dependencias **faltantes** a `pom.xml` de `sga-jee-web`.

✦ Contenido actualizado de `pom.xml`:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>sga</groupId>
    <artifactId>sga-jee-web</artifactId>
    <version>1.0</version>
    <packaging>war</packaging>
    <name>sga-jee-web</name>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <jakartaee>11.0.0-M1</jakartaee>
    </properties>

    <dependencies>
        <dependency>
            <groupId>jakarta.platform</groupId>
            <artifactId>jakarta.jakartaee-api</artifactId>
            <version>${jakartaee}</version>
            <scope>provided</scope>
        </dependency>
        <dependency>
            <groupId>jakarta.persistence</groupId>
            <artifactId>jakarta.persistence-api</artifactId>
            <version>3.2.0</version>
        </dependency>
        <dependency>
            <groupId>org.eclipse.persistence</groupId>
            <artifactId>eclipselink</artifactId>
            <version>4.0.5</version>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>9.2.0</version>
        </dependency>
    </dependencies>
```

```
<groupId>org.slf4j</groupId>
<artifactId>slf4j-api</artifactId>
<version>2.0.16</version>
<type>jar</type>
</dependency>
<dependency>
<groupId>org.slf4j</groupId>
<artifactId>slf4j-simple</artifactId>
<version>2.0.16</version>
<scope>runtime</scope>
</dependency>
</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.12.1</version>
<configuration>
<source>21</source>
<target>21</target>
</configuration>
</plugin>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-war-plugin</artifactId>
<version>3.4.0</version>
</plugin>
</plugins>
</build>
</project>
```

✓ **Nuevas dependencias agregadas:**

- **JPA** (Jakarta Persistence API).
- **EclipseLink** (Implementación de JPA).
- **Conector de MySQL**.
- **SLF4J** (Logging).

6 Construir el Proyecto

Después de modificar `pom.xml`, generamos los archivos necesarios:

1. **Click derecho** sobre `sga-jee-web`.
2. Seleccionar **Clean and Build**.
3. Esperar a que **se descarguen las dependencias y se compile correctamente**.

7 Implementar la Capa DAO

Ahora implementamos el **patrón DAO** para acceder a la base de datos.

✦ Crear PersonaDao.java:

```
package sga.datos;

import java.util.List;
import sga.domain.Persona;

public interface PersonaDao {
    public List<Persona> findAllPersonas();

    public Persona findPersonaById(Persona persona);

    public Persona findPersonaByEmail(Persona persona);

    public void insertPersona(Persona persona);

    public void updatePersona(Persona persona);

    public void deletePersona(Persona persona);
}
```

✦ Crear PersonaDaoImpl.java:

```
package sga.datos;

import java.util.List;
import jakarta.ejb.Stateless;
import jakarta.persistence.*;
import sga.domain.Persona;

@Stateless
public class PersonaDaoImpl implements PersonaDao{

    @PersistenceContext(unitName="PersonaPU")
    EntityManager em;

    @Override
    public List<Persona> findAllPersonas() {
        return em.createNamedQuery("Persona.findAll").getResultList();
    }

    @Override
    public Persona findPersonaById(Persona persona) {
        return em.find(Persona.class, persona.getIdPersona());
    }

    @Override
    public Persona findPersonaByEmail(Persona persona) {
        Query query = em.createQuery("from Persona p where p.email =: email");
        query.setParameter("email", persona.getEmail());
        return (Persona) query.getSingleResult();
    }

    @Override
    public void insertPersona(Persona persona) {
        em.persist(persona);
    }
}
```

```

    }

    @Override
    public void updatePersona(Persona persona) {
        em.merge(persona);
    }

    @Override
    public void deletePersona(Persona persona) {
        em.remove(em.merge(persona));
    }
}

```

8 Modificar el archivo persistence.xml

Ahora modificaremos el archivo persistence.xml para configurar el manejo de JTA y Glassfish.

Modificar persistence.xml:

```

<?xml version="1.0" encoding="UTF-8"?>
<persistence version="3.2"
    xmlns="https://jakarta.ee/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
https://jakarta.ee/xml/ns/persistence/persistence_3_2.xsd">
    <persistence-unit name="PersonaPU" transaction-type="JTA">
        <jta-data-source>jdbc/PersonaDb</jta-data-source>
        <!--
        <class>sga.domain.Persona</class>
        <properties>
            <property name="jakarta.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/test?useSSL=false&useTimezone=true&serverTimez
one=UTC" />
            <property name="jakarta.persistence.jdbc.user" value="root" />
            <property name="jakarta.persistence.jdbc.password" value="admin" />
            <property name="jakarta.persistence.jdbc.driver"
value="com.mysql.cj.jdbc.Driver" />
            <property name="eclipselink.logging.level.sql" value="FINE" />
            <property name="eclipselink.logging.parameters" value="true" />
            <property name="eclipselink.logging.timestamp" value="false"/>
        </properties>-->
    </persistence-unit>
</persistence>

```

Hasta Aquí la Guía

-  Se configuró sga-jee-web para usar EJBs y JPA.
-  Se creó la capa de datos DAO.

✓ Se actualizó la configuración del archivo persistence.xml.

✚ **En la siguiente lección del curso**, configuraremos **GlassFish** y **JTA** para la conexión a la base de datos.

🎉 ¡Felicidades! Has configurado tu proyecto SGA Web para soportar JPA y la capa de datos aplicando el patrón de diseño DAO con Jakarta EE! 🚀

🎯 Próximos Pasos (En los siguientes videos)

- ◆ Configurar `GlassFish` para manejar transacciones con **JTA**.
- ◆ Implementar **la capa de servicio completa**.
- ◆ Desplegar y probar la aplicación en un servidor Jakarta EE.

El código fuente completo lo puedes descargar en la sección de recursos de este video.

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)